Лабораторна робота № 2

Виконала: Кісельгова Ксенія ТТП-31

Досліджувана програма

Програма на C++ послідовно виконує 100 млн обчислень синуса та 100 млн обчислень квадратного кореня, підсумовує результати та вимірює час виконання за допомогою std::chrono::steady_clock. Атрибут noinline зберігає окремі функції для профілювання. Змінна volatile обмежує деякі оптимізації, що важливо врахувати в аналізі.

```
#include <iostream>
#include <cmath>
#include <chrono>

constexpr int N = 100000000;

__attribute__((noinline))
double calculateSin() {
    volatile double result = 0;
    for (int i = 1; i <= N; ++i) {
        result = result + std::sin(i * 0.001);
    }
    return result;
}

__attribute__((noinline))
double calculateSqrt() {
    volatile double result = 0;
    for (int i = 1; i <= N; ++i) {
        result = result + std::sqrt(i);
    }
    return result;
}

__attribute__((noinline))
double processData() {
    double a = calculateSin();
```

```
    double b = calculateSqrt();
    return a + b;
}


int main() {
    auto start = std::chrono::steady_clock::now();
    double result = processData();
    auto end = std::chrono::steady_clock::now();
    std::chrono::duration<double> elapsed = end - start;
    std::cout << "Result: " << result << '\n';
    std::cout << "Time: " << elapsed.count() << " s\n";
    return 0;
}
```

1. Побудова FlameGraph

1.1. Демонстрація процесу побудови

Для програми без оптимізації зібрано профіль виконання з подією cpu-clock; після збільшення кількості ітерацій зафіксовано 137 зразків. Отриманий файл perf.data перетворено на складені стеки викликів, а далі — на SVG-графік.

```
sudo perf record -F 99 -g -e cpu-clock -- ./program
sudo perf script > perf.out
./FlameGraph/stackcollapse-perf.pl perf.out > perf.folded
./FlameGraph/flamegraph.pl perf.folded > flamegraph.svg
```

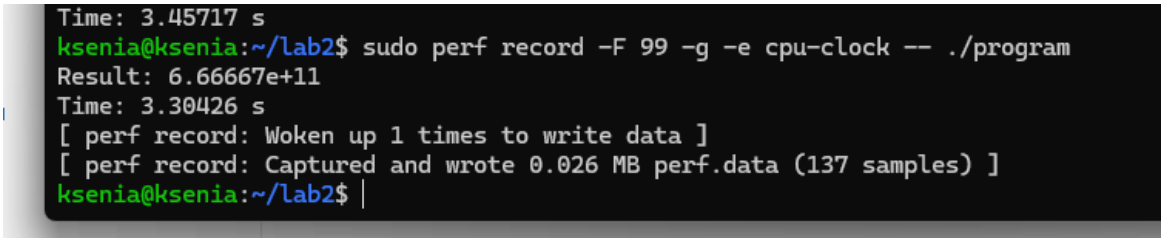


Рис. 1 — запис профілю виконання (137 зразків).

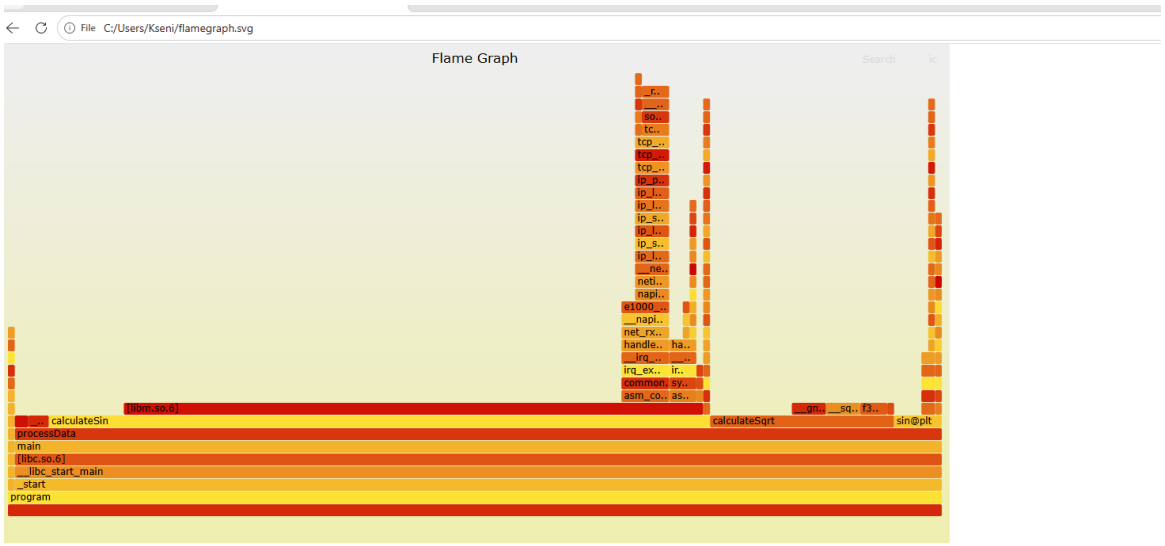


Рис. 2 — FlameGraph для версії -O0.

1.2. Інтерпретація FlameGraph

Основний шлях викликів: program → main → processData → calculateSin / calculateSqrt. Ширина прямокутників відображає частку відібраних стеків, у яких присутня функція; висота — глибину вкладеності викликів, а не тривалість. У профілі домінує гілка calculateSin із викликами математичної бібліотеки libm.so.6. Гілка calculateSqrt помітно менша. Кольори служать переважно для візуального розрізнення блоків, а не відображають швидкість або енергоспоживання. Через обмежену кількість зразків результати мають вибірковий характер.

2. Статистика виконання

2.1. /usr/bin/time --verbose

```
/usr/bin/time --verbose ./program
```

Показник (VirtualBox, -O0)	Значення
Elapsed time	2,87 с
User time	2,43 с
System time	0,44 с
CPU	99 %
Maximum resident set size	4528 КБ
Minor page faults	159
Context switches	2 (1 добровільне, 1 примусове)
Exit status	0

Програма має переважно обчислювальний характер: протягом виконання майже повністю використовувала одне логічне ядро процесора. Максимальне використання пам’яті — близько 4,4 МБ. Код завершення 0 означає успішне виконання.

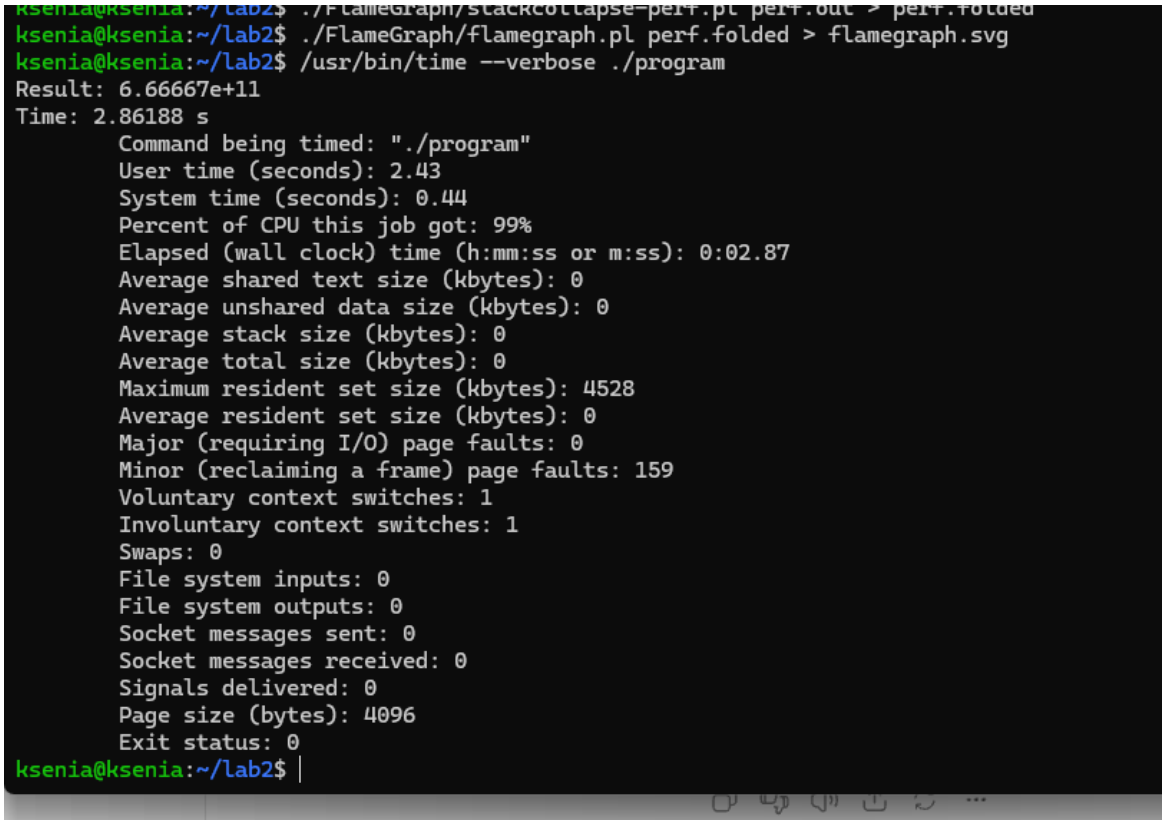


Рис. 3 — результати /usr/bin/time --verbose (-O0).

2.2. perf stat -d

Апаратну статистика для досліджуваної програми; вимірювання -O0 наведена нижче.

```
sudo perf stat -d -- ~/lab2/program_O0
```

Показник (фізичний Linux, -O0)	Значення
Elapsed time	4,170 с
Task clock	4169,15 мс
Cycles	12 021 781 032
Instructions	16 713 067 298
IPC	1,39
Branches	2 354 143 732
Branch misses	576 698 (0,02 %)
L1-dcache-load-misses	376 828

LLC-load-misses	4 953

IPC (instructions per cycle) показує середню кількість інструкцій, виконаних за такт. Показник 1,39 означає близько 1,39 інструкції за такт у цьому запуску. Частота промахів прогнозування переходів невелика, оскільки цикли мають передбачувану структуру. Частина апаратних показників збиралася з мультиплексуванням (відсотки часу роботи лічильників у дужках), тому значення є оцінками perf.

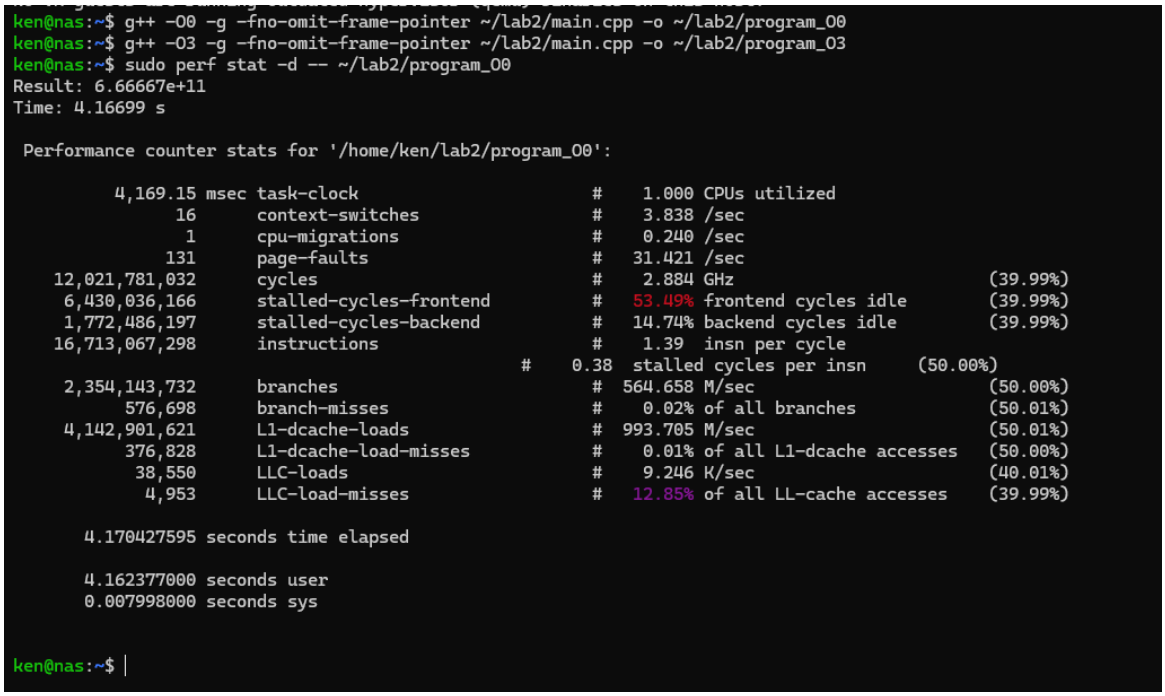


Рис. 4 — perf stat -d для -O0

2.3. perf report (після perf record)

```
sudo perf report --stdio --no-children
```

За результатами вибіркового профілювання в середовищі VirtualBox зібрано 137 зразків, втрачено 0. Серед видимих власних часток (self overhead): calculateSqrt() — 8,76 %, calculateSin() — 8,03 %. Параметр --no-children показує власну вартість функції без витрат її дочірніх викликів, тому ці числа не тотожні ширині гілок FlameGraph. Значну частку витрат calculateSin становить виклик бібліотечної функції sin.

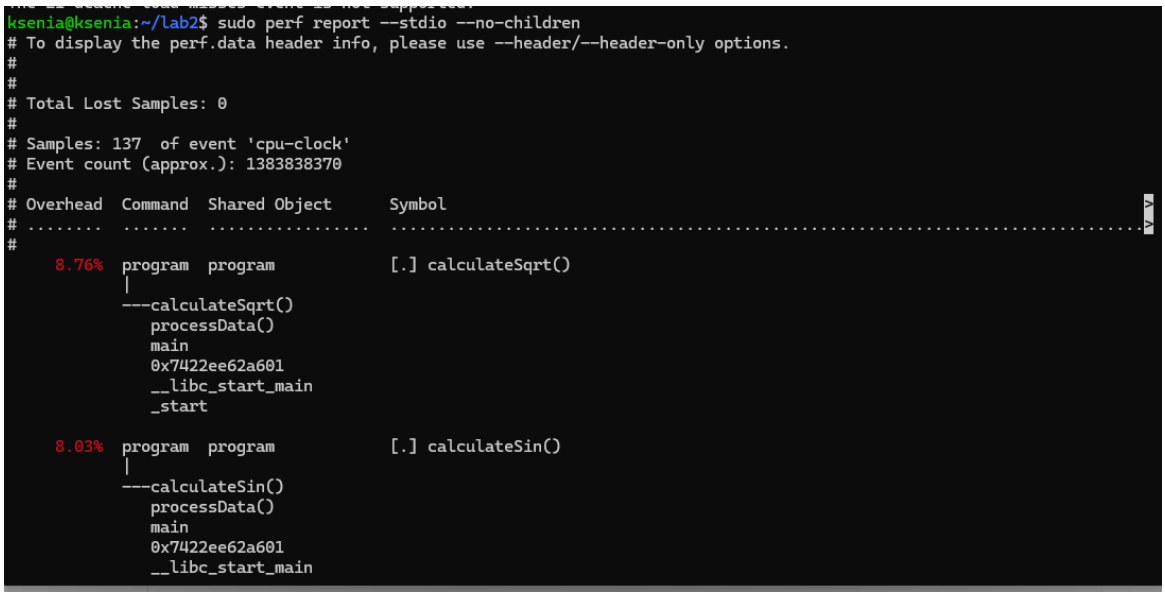


Рис. 5 — частина perf report з власними частками функцій.

3. Вимірювання енергоспоживання

3.1. Енергоспоживання системи

Вимірювання виконано 26.10.2026 у Windows PowerShell через WMI-клас BatteryStatus (поле DischargeRate) за умови роботи ноутбука від акумулятора. Програму запускали у WSL2; показання стосуються всього ноутбука, а не окремого процесу.

Для кожного режиму збережено 20 відліків орієнтовно через 2 с; потужність оцінено як DischargeRate / 1000 (Вт). Показання змінюються ступінчасто, тому їх не можна вважати незалежними вимірюваннями з інтервалом 2 с.

Режим / серія	Серія 1, Вт	Серія 2, Вт
Простій (Idle)	35,382	не вимірювався
Програма без оптимізації (-O0)	42,086	38,141
Програма з оптимізацією (-O3)	38,076	41,599

Таблиця 1 — Середні показання потужності розряду батареї.

Перша серія (Idle → -O0 → -O3) показала зменшення середньої потужності у режимі -O3 на 4,010 Вт (9,5%) порівняно з -O0. Повторна серія у зворотному порядку також дала реальні показання, але з іншою тенденцією. Це демонструє залежність вимірювань від умов експерименту.

3.2. Енергоспоживання досліджуваної програми

Для наближеної оцінки додаткової потужності можна відняти базовий рівень:

\[P_{\text{дод}}=P_{\text{режиму}}-P_{\text{фон}} \]

Проте фон виміряно лише в першій серії, а сама потужність охоплює систему повністю.

Оцінка за першою серією	-O0	-O3
Середня потужність системи, Вт	42,086	38,076
Фонова потужність, Вт	35,382	35,382
Додаткова потужність, Вт	6,704	2,694

Таблиця 2 — Орієнтовна додаткова потужність за першою серією.

Для оцінки використовуємо формулу:

\[E_{\text{дод}}\approx P_{\text{дод}}\times t \]

За першою серією різниця додаткової потужності між -O0 та -O3 становила 4,010 Вт. Оскільки відліки батареї оновлюються із затримкою, енергію окремого запуску в джоулях за цими даними точно не визначали.

4. Порівняння до та після оптимізації

4.1. Порівняння асемблерного коду

g++ -O0 -S -masm=intel main.cpp -o main_O0.s

g++ -O3 -S -masm=intel main.cpp -o main_O3.s

У фрагменті calculateSin для -O0 лічильник циклу зберігається у стеку за адресою [rbp-12], а його значення багаторазово завантажується та змінюється в пам’яті. У версії -O3 лічильник знаходиться у регістрі ebx та змінюється інструкцією add ebx, 1. Компілятор також змінює структуру циклу: порівняння і переходи jle/jne. Виклик sin@PLT зберігається; змінна volatile не дозволяє усунути всі операції читання та запису накопичувача.

```
+       .cfi_offset 3, -16
+       mov     ebx, 2
        sub     rsp, 16
ksenia@ksenia:~/lab2$ sed -n '/^_Z12calculateSinv:/,/^[[:space:]]*\.size[[:space:]]*_Z12calculateSinv/p' main_O0.s | head -65
_Z12calculateSinv:
.LFB2708:
        .cfi_startproc
        endbr64
        push    rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        mov     rbp, rsp
        .cfi_def_cfa_register 6
        sub     rsp, 16
        pxor    xmm0, xmm0
        movsd   QWORD PTR -8[rbp], xmm0
        mov     DWORD PTR -12[rbp], 1
        jmp     .L5
.L6:
        pxor    xmm1, xmm1
        cvtsi2sd        xmm1, DWORD PTR -12[rbp]
        movsd   xmm0, QWORD PTR .LC1[rip]
        mulsd   xmm1, xmm0
        movq    rax, xmm1
        movq    xmm0, rax
        call    sin@PLT
        movsd   xmm1, QWORD PTR -8[rbp]
        addsd   xmm0, xmm1
        movsd   QWORD PTR -8[rbp], xmm0
        add     DWORD PTR -12[rbp], 1
.L5:
        cmp     DWORD PTR -12[rbp], 100000000
        jle     .L6
        movsd   xmm0, QWORD PTR -8[rbp]
        leave
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc
.LFE2708:
        .size   _Z12calculateSinv, .-_Z12calculateSinv
ksenia@ksenia:~/lab2$
```

Рис. 6 — фрагмент calculateSin у -O0.

```
ksenia@ksenia:~/lab2$ sed -n '/^_Z12calculateSinv:/,/^[[:space:]]*\.size[[:space:]]*_Z12calculateSinv/p' main_O3.s | head -65
_Z12calculateSinv:
.LFB2797:
        .cfi_startproc
        endbr64
        push    rbx
        .cfi_def_cfa_offset 16
        .cfi_offset 3, -16
        mov     ebx, 2
        sub     rsp, 16
        .cfi_def_cfa_offset 32
        mov     QWORD PTR 8[rsp], 0x000000000
        movsd   xmm0, QWORD PTR 8[rsp]
        addsd   xmm0, QWORD PTR .LC1[rip]
        movsd   QWORD PTR 8[rsp], xmm0
        .p2align 4
        .p2align 3
.L2:
        pxor    xmm0, xmm0
        cvtsi2sd        xmm0, ebx
        mulsd   xmm0, QWORD PTR .LC2[rip]
        add     ebx, 1
        call    sin@PLT
        movapd  xmm1, xmm0
        movsd   xmm0, QWORD PTR 8[rsp]
        addsd   xmm0, xmm1
        movsd   QWORD PTR 8[rsp], xmm0
        cmp     ebx, 100000001
        jne     .L2
        movsd   xmm0, QWORD PTR 8[rsp]
        add     rsp, 16
        .cfi_def_cfa_offset 16
        pop     rbx
        .cfi_def_cfa_offset 8
        ret
        .cfi_endproc
.LFE2797:
        .size   _Z12calculateSinv, .-_Z12calculateSinv
ksenia@ksenia:~/lab2$
```

Рис. 7 — фрагмент calculateSin у -O3.

4.1.1. Інструкції AVX/SIMD

Пошук інструкцій роботи з векторними регістрами ymm/zmm і характерних vaddpd/vmulpd/vsqrtpd у main_O3.s збігів не дав. У показаному фрагменті calculateSin використовуються скалярні інструкції на регістрах xmm (зокрема mulsd та addsd), але це не є доказом векторизації кількох елементів одночасно.

Виклики sin та volatile обмежують можливості автовекторизації. Тому прискорення не варто приписувати SIMD без додаткового підтвердження.

4.2. Порівняння часу та характеристик виконання

Для зіставності використано вимірювання perf stat -d обох програм.

Показник	-O0	-O3
Час виконання	4,170 с	3,913 с
Cycles	12 021 781 032	11 224 565 073
Instructions	16 713 067 298	14 508 036 928
IPC	1,39	1,29
Branch misses	576 698	581 893
L1 cache misses	376 828	392 566
LLC cache misses	4 953	320

Час скоротився приблизно на 6,2 %, кількість інструкцій — на 13,2 %, тактів — на 6,6 %. IPC дещо знизився, однак загальна кількість інструкцій і тактів стала меншою, тож програма виконалася швидше.

```
ken@nas:~$ sudo perf stat -d -- ~/lab2/program_O3
Result: 6.66667e+11
Time: 3.89188 s

 Performance counter stats for '/home/ken/lab2/program_O3':

          3,894.33 msec task-clock                       #    0.995 CPUs utilized
                14      context-switches                 #    3.595 /sec
                 0      cpu-migrations                   #    0.000 /sec
               133      page-faults                      #   34.152 /sec
    11,224,565,073      cycles                           #    2.882 GHz                         (39.97%)
     6,484,491,878      stalled-cycles-frontend          #   57.77% frontend cycles idle        (40.00%)
     3,011,512,241      stalled-cycles-backend           #   26.83% backend cycles idle         (40.02%)
    14,508,036,928      instructions                     #    1.29  insn per cycle
                                                  #    0.45  stalled cycles per insn     (50.04%)
     1,753,769,379      branches                         #  450.339 M/sec                       (50.06%)
           581,893      branch-misses                    #    0.03% of all branches             (50.04%)
     3,046,229,218      L1-dcache-loads                  #  782.221 M/sec                       (50.01%)
           392,566      L1-dcache-load-misses            #    0.01% of all L1-dcache accesses   (49.99%)
            37,310      LLC-loads                        #    9.581 K/sec                       (39.95%)
               320      LLC-load-misses                  #    0.86% of all LL-cache accesses    (39.95%)

       3.912988802 seconds time elapsed

       3.894451000 seconds user
       0.001000000 seconds sys
```

Рис. 8 — perf stat -d для -O3

Додатково через /usr/bin/time --verbose у VirtualBox отримано 2,87 с (-O0) та 2,53 с (-O3), що відповідає скороченню часу приблизно на 11,8 %. Це окрема серія вимірювань, її не об’єднано з результатами фізичного комп’ютера.

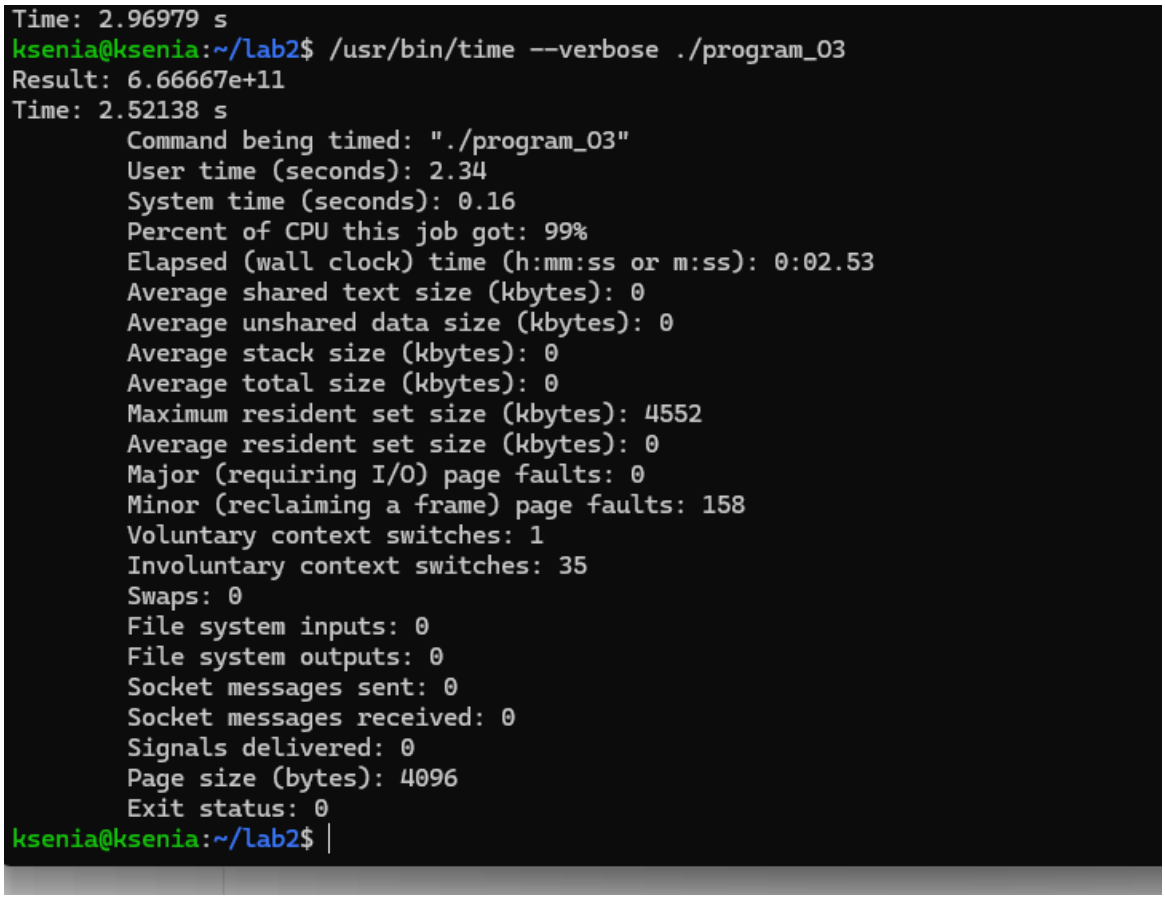


Рис. 9 — /usr/bin/time --verbose після оптимізації.

4.3. Порівняння FlameGraph

```
sudo perf record -F 999 -g -e cpu-clock -o perf_O3_better.data -- ./program_O3
sudo perf script -i perf_O3_better.data > perf_O3_better.out
./FlameGraph/stackcollapse-perf.pl perf_O3_better.out > perf_O3_better.folded
./FlameGraph/flamegraph.pl perf_O3_better.folded > flamegraph_O3_better.svg
```

У базовому FlameGraph (-O0, 137 зразків) та оптимізованому (-O3, 454 зразки) домінує гілка calculateSin, a calculateSqrt має меншу частку. Оптимізований графік візуально містить менше сторонніх системних стеків, проте це не слід безпосередньо трактувати як зменшення часу системних операцій: збирання відбувалося з різними частотами та різною кількістю зразків. Графіки показують розподіл обчислювальної активності, а кількісне прискорення підтверджують часові вимірювання.

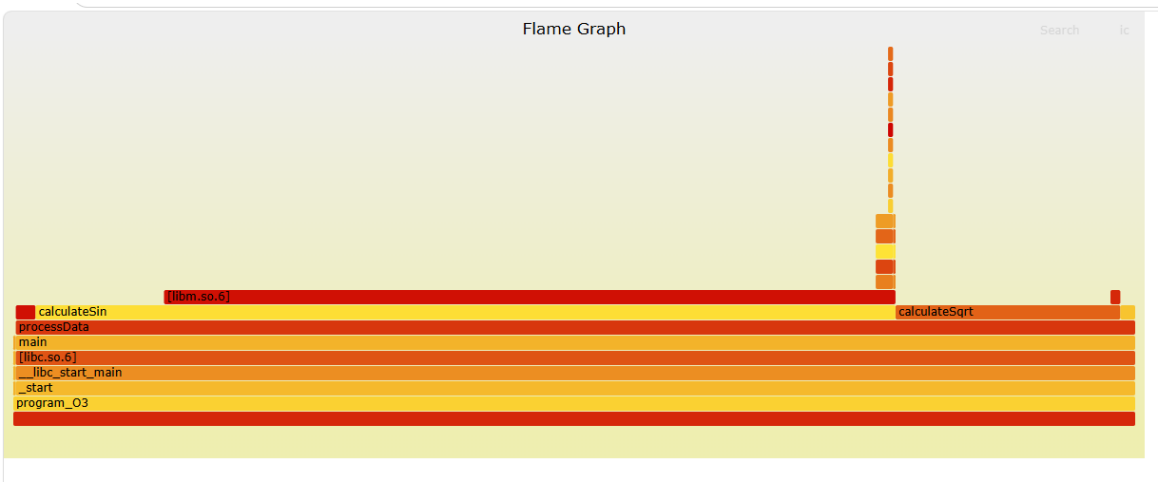


Рис. 10 — FlameGraph програми -O3 (454 зразки).

Для версії -O3 частоту профілювання було збільшено з 99 до 999 Гц, оскільки початкова кількість зразків була недостатньою для детального відображення функцій на FlameGraph.

4.4. Порівняння енергоспоживання

Середня потужність, Вт	**-O0**	**-O3**
Серія 1	42,086	38,076
Серія 2 (зворотний порядок)	38,141	41,599
Середнє двох серій*	40,114	39,838

Таблиця 3 — Порівняння середньої потужності до та після оптимізації.

Середнє двох серій наведено описово; розкид між серіями суттєвий.

У першій серії під час виконання -O3 зафіксовано середню потужність 38,076 Вт проти 42,086 Вт для -O0, тобто на 4,010 Вт (9,5%) менше. Повторна серія у зворотному порядку показала 41,599 Вт для -O3 та 38,141 Вт для -O0. Отже, вимірювання виконано успішно, але їх результати залежать від умов запуску.

Умови та обмеження вимірювання

Джерело — WMI BatteryStatus.DischargeRate, ноутбук без зовнішнього живлення. Запуски -O0 та -O3 виконувалися серіями через WSL2, а відліки робилися приблизно кожні 2 секунди. Датчик оновлював значення з затримкою; крім програми, на розряд впливали екран, фонові процеси, температура й інше навантаження.

Апаратного лічильника енергії окремого процесу (RAPL) не було доступно, тому отримані значення описують потужність розряду ноутбука. Дані підтверджують можливість практичного контролю енергоспоживання, проте не дають підстав стверджувати стабільну економію енергії -O3 у кожному запуску.

Висновок

Засобами time, perf і FlameGraph досліджено виконання програми C++ та підтверджено прискорення після оптимізації -O3. Також за допомогою Windows PowerShell і WMI BatteryStatus виконано дві серії реальних спостережень потужності розряду акумулятора. У першій серії -O3 показала меншу середню потужність на 9,5%; у повторній серії різниця змінила знак. Отже, експеримент демонструє роботу методики та важливість контролю умов вимірювання.